

Zadanie

SIMPLECHAT_VT

Napisać serwer czatu, który:

- obsługuje logowanie klientów (tylko id, bez hasła),
- przyjmuje od klientów wiadomości i rozsyła je do zalogowanych klientów,
- obsługuje wylogowanie klientów,
- gromadzi wszystkie odpowiedzi na żądania klientów w logu, realizowanym w pamięci wewnętrznej (poza systemem plikowym). W logu powinna się znaleźć również informacja o zakończeniu czatu (ChatServer: chat closed), która ma pojawić się po zatrzymaniu serwera.

Zadania te wykonuje klasa **ChatServer**, która ma:

- konstruktor: **public ChatServer(int port)**
- metodę: **public void startServer()**, która uruchamia serwer w odrębnym wątku,
- metodę: **public void stopServer()**, która zatrzymuje serwer i wątek, w którym działa,
- metodę **String getServerLog()** - zwraca log serwera (wymagany format logu będzie widoczny w dalszych przykładach).

Wymagania konstrukcyjne dla klasy ChatServer:

- **obsługa żądań klientów odbywa się w wirtualnych wątkach**

Dostarczyć także klasy **ChatClient** z konstruktorem:

public ChatClient(String host, int port, String id), gdzie id - id klienta

i następującymi metodami:

- **public void login()** - loguje klienta do czatu

- **public void logout()** - wylogowuje klienta,
- **public void send(String req)** - wysyła do serwera żądanie req
- **public String getChatView()** - zwraca dotychczasowy widok czatu z pozycji danego klienta (czyli wszystkie informacje, jakie dostaje on po kolei od serwera)

Dla metody *send* żądaniem może być posłanie tekstu wiadomości, zalogowanie, wylogowanie, a protokół komunikacji z serwerem można po swojemu wymyślić.

Dodatkowo stworzyć klasę **ChatClientTask**, umożliwiającą uruchamianie klientów w odrębnych wątkach poprzez *ExecutorService*.

Obiekty tej klasy tworzy statyczna metoda:

public static ChatClientTask create(ChatClient c, List<String> msgs, int wait)

gdzie:

c - klient (obiekt klasy ChatClient)

msgs - lista wiadomości do wysłania przez klienta c

wait - czas wstrzymania pomiędzy wysyłaniem żądań.

Kod działający w wątku ma wykonywać następując działania:

- łączy się z serwerem i loguje się (c.login())
- wysyła kolejne wiadomości z listy msgs (c.send(...))
- wylogowuje klienta (c.logout())

Parametr *wait* w sygnaturze metody *create* oznacza czas w milisekundach, na jaki wątek danego klienta jest wstrzymywany po każdym żądaniu. Jeśli *wait* jest 0, wątek klienta nie jest wstrzymywany,

Oto pseudokod fragmentu odpowiedzialnego za posyłanie żądań:

```
c.login();
if (wait != 0) uśpienie_watku_na wait ms;
// ....
dla_każdej_wiadomości_z_listy msgs {
    // ...
    c.send( wiadomość );
    if (wait != 0) uśpienie_watku_na wait ms;
```

```
}  
c.logout();  
if (wait != 0) uśpienie_watku_na wait ms;
```

W projekcie znajduje się klasa **MainChatVT (plik niemodyfikowalny)**, w której z pliku testowego (ChatTestVT.yaml z katalogu user.home) wprowadzane są informacje nt. opcji działania.

Format pliku YAML:

pod kluczem **portAndDelays** znajduje się lista liczb całkowitych o postaci:

```
[ port, startTaskDelay, stopServerDelay ]
```

gdzie:

- port - numer portu serwera,
- startTaskDelay - opóźnienie przed startem kolejnego zadania klienta (w ms)
- stopServerDelay - opóźnienie przed zatrzymaniem serwera.

Parametry te są używane w metodzie main i tam widac ich działanie.

Dalej w pliku testowym znajdują się informacje o klientach w postaci wejść w mapie:

```
id_klienta: [ opóźnienie_po_wysłaniu_żądania (wait), liczba_posyłanych_wiadomości]
```

Na przykład takie opcje:

```
portAndDelays: [ 7799, 0, 50]
```

```
Asia: [30, 3]
```

```
Adam: [30, 3]
```

```
Sara: [30, 3]
```

oznacza, że server działa na porcie 7799, klienci startują równocześnie (opóźnienie 0), przed zatrzymaniem serwera główny wątek przysypia na 50 ms.

I mamy trzech klientów (Asia, Adam, Sara), każdy wysyła 3 wiadomości i po każdym żądaniu przysypia na 30 ms.

Uwaga: wiadomości są automatycznie generowane i mają postać: msg_1. msg_2 itd.

Klasa MainChatVT:

```

import java.nio.charset.*;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;

import org.yaml.snakeyaml.*;

public class MainChatVT {

    public static Map<String, String> doChat( Map<String, List<Integer>> opts) throws
    InterruptedException {
        List<Integer> pd = opts.remove("portAndDelays");
        int port = pd.get(0);
        int startTaskDelay = pd.get(1);
        int stopServerDelay = pd.get(2);

        ChatServer s = new ChatServer(port);
        s.startServer();

        ExecutorService es = Executors.newCachedThreadPool();
        List<ChatClientTask> ctasks = new ArrayList<>();

        // Creating and starting clients
        opts.forEach((k, v) -> {
            String cid = k;
            int wait = v.get(0);
            List<String> msgs = IntStream.rangeClosed(1, v.get(1)).mapToObj(n ->
"msg_" + n).toList();
            ChatClient c = new ChatClient("localhost", port, cid);
            ChatClientTask ctask = ChatClientTask.create(c, msgs, wait);
            ctasks.add(ctask);
            if (startTaskDelay > 0)
                try {
                    Thread.sleep(startTaskDelay);
                } catch (InterruptedException exc) {
                    throw new RuntimeException(exc);
                }
            es.execute(ctask);
        });
    }

```

```

ctasks.forEach( task -> {
    try {
        task.get();
    } catch (InterruptedException | ExecutionException exc) {
        System.out.println("*** " + exc);
    }
});
if (stopServerDelay > 0) Thread.sleep(stopServerDelay);
s.stopServer();
es.shutdown();
Thread.sleep(100);

```

```

Map<String, String> results = new LinkedHashMap<>();
results.put("#server#", s.getServerLog());
ctasks.forEach(task -> {
    ChatClient c = task.getClient();
    results.put(c.getId(), c.getChatView());
});
return results;
}

```

```

public static void main(String[] args) throws Exception {
    String testFileName = System.getProperty("user.home") + "/ChatTestVT.yaml";
    String options = Files.readString(Path.of(testFileName), Charset.forName("UTF-8"));
    System.out.println("=== Options\n" + options);
}

```

```

Map<String, List<Integer>> imap = (Map<String, List<Integer>>) new
Yaml().load(options);
doChat(imap).forEach ( (k, v) -> {
    if (k.equals("#server#")) System.out.println("\n=== Server log\n" + v);
    else System.out.println("=== " + k + " chat view\n" + v);
});
}
}

```

Dla podanego pliku testowego program ten może wyprowadzić:

=== Options

portAndDelays: [7799, 0, 50]

Asia: [30, 3]

Adam: [30, 3]

Sara: [30, 3]

Server started

Server stopped

=== Server log

17:34:29.593409 Asia logged in

17:34:29.600410400 Adam logged in

17:34:29.600410400 Sara logged in

17:34:29.611412600 Sara: msg_1

17:34:29.611412600 Adam: msg_1

17:34:29.611412600 Asia: msg_1

17:34:29.642851700 Adam: msg_2

17:34:29.642851700 Asia: msg_2

17:34:29.642851700 Sara: msg_2

17:34:29.673629700 Sara: msg_3

17:34:29.673629700 Asia: msg_3

17:34:29.673629700 Adam: msg_3

17:34:29.704426400 Asia logged out

17:34:29.704426400 Adam logged out

17:34:29.704426400 Sara logged out

17:34:29.798997800 ChatServer: chat closed

=== Asia chat view

Asia logged in

Adam logged in

Sara logged in

Sara: msg_1

Adam: msg_1

Asia: msg_1

Adam: msg_2

Asia: msg_2

Sara: msg_2

Sara: msg_3

Asia: msg_3

Adam: msg_3

Asia logged out

=== Adam chat view

Adam logged in

Sara logged in

Sara: msg_1

Adam: msg_1

Asia: msg_1

Adam: msg_2

Asia: msg_2

Sara: msg_2

Sara: msg_3

Asia: msg_3

Adam: msg_3

Asia logged out

Adam logged out

=== Sara chat view

Sara logged in

Sara: msg_1

Adam: msg_1

Asia: msg_1

Adam: msg_2

Asia: msg_2

Sara: msg_2

Sara: msg_3

Asia: msg_3

Adam: msg_3

Asia logged out

Adam logged out

Sara logged out

Przykład pokazuje wymaganą formę wydruku (chatView klienta, wejścia w logu serwera).
W szczególności:

- start serwera powoduje wypisanie na konsoli: Server started
- logowanie klienta *id* skutkuje rozestaniem wiadomości: ***id* logged in**
- wylogowanie klienta *id* skutkuje rozestaniem wiadomości: ***id* logged out**
- otrzymanie wiadomości *msg* od klienta *id* skutkuje rozesłaniem wiadomości ***id: msg***
- log serwera zawiera kolejno wszystkie odpowiedzi serwera z podaniem czasu w formacie HH:MM:SS.nnn, gdzie nnn - nanosekundy,
- zatrzymanie serwera wypisuje na konsoli: Server stopped
- zatrzymanie serwera powoduje rozestanie do **zalogowanych** klientów wiadomości: "ChatServer: chat closed" i taka wiadomość powinna pojawić się w chatView zalogowanych w tym momencie klientów.

Forma wyniku jest obowiązkowa.

Konkretna zawartość wydruków chatview i logu serwera (kolejność wierszy, podane czasy itp.) mogą być w każdym przebiegu inne (i będą inne też przy różnych opcjach - opóźnieniach), ważne jednak, aby widac było równoległą obsługę klientów oraz zachowaną logikę: klienci otrzymują, w odpowiedniej kolejności, tylko te wiadomości, które się pojawiły od momentu ich logowania do momentu wylogowania.

Podsumowanie:

trzeba stworzyć klasy ChatServer, ChatClient, ChatClientTask w taki sposób, aby zapewnić właściwe wykonanie kodu metody main z klasy MainChatVT, przy czym w obsłudze żądań klientów należy użyć wirtualnych wątków.

Ale:

trzeba przygotować kod na rozliczne konfiguracje podawane w pliku ChatTestVT.yaml.
Przykłady wyników:

Przykład 2. Klienci bez opóźnień

=== Options

portAndDelays: [7899, 0, 50]

Asia: [0, 3]

Adam: [0, 3]

Sara: [0, 3]

Server started

Server stopped

=== Server log

18:35:03.912118400 Sara logged in

18:35:03.919118800 Adam logged in

18:35:03.920120500 Asia logged in

18:35:04.007126500 Asia: msg_1

18:35:04.008132 Sara: msg_1

18:35:04.008132 Adam: msg_1
18:35:04.008132 Asia: msg_2
18:35:04.009131500 Asia: msg_3
18:35:04.009131500 Sara: msg_2
18:35:04.009131500 Adam: msg_2
18:35:04.010131400 Asia logged out
18:35:04.010131400 Sara: msg_3
18:35:04.010131400 Adam: msg_3
18:35:04.011129800 Sara logged out
18:35:04.011129800 Adam logged out
18:35:04.068743700 ChatServer: chat closed

=== Asia chat view

Asia logged in
Asia: msg_1
Sara: msg_1
Adam: msg_1
Asia: msg_2
Asia: msg_3
Sara: msg_2
Adam: msg_2
Asia logged out

=== Adam chat view

Adam logged in
Asia logged in
Asia: msg_1
Sara: msg_1

Adam: msg_1

Asia: msg_2

Asia: msg_3

Sara: msg_2

Adam: msg_2

Asia logged out

Sara: msg_3

Adam: msg_3

Sara logged out

Adam logged out

=== Sara chat view

Sara logged in

Adam logged in

Asia logged in

Asia: msg_1

Sara: msg_1

Adam: msg_1

Asia: msg_2

Asia: msg_3

Sara: msg_2

Adam: msg_2

Asia logged out

Sara: msg_3

Adam: msg_3

Sara logged out

Przykład 3 (brak jakichkolwiek opóźnień - wynik może być bardzo różny).

=== Options

portAndDelays: [9999, 0, 0]

Asia: [0, 3]

Adam: [0, 3]

Sara: [0, 3]

Server started

Server stopped

=== Server log

18:43:49.442937 Asia logged in

18:43:49.449938500 Sara logged in

18:43:49.450938600 Adam logged in

18:43:49.534949100 Asia: msg_1

18:43:49.535949200 Adam: msg_1

18:43:49.535949200 Sara: msg_1

18:43:49.535949200 ChatServer: chat closed

=== Asia chat view

Asia logged in

Sara logged in

Adam logged in

Asia: msg_1

Adam: msg_1

Sara: msg_1

ChatServer: chat closed

=== Adam chat view

Adam logged in

Asia: msg_1

Adam: msg_1

Sara: msg_1

ChatServer: chat closed

=== Sara chat view

Sara logged in

Adam logged in

Asia: msg_1

Adam: msg_1

Sara: msg_1

ChatServer: chat closed